WEB APPLICATION SECURITY

CSRF
Security Assessment

Account management features

Email change & password change

Assessment of token validation, cross-account rejection, token lifecycle and current-password enforcement.

DOCUMENT CONTROL	
Prepared by	Haroon Shahid
Report reference	CSRF-2026-01
Issued	7 November 2026
Version	1.1 / presentation revision
Edition	Anonymized; target and credentials withheld

ASSESSMENT OUTCOME

No CSRF vulnerability identified in the tested scenarios.

REPORT CONTENTS

01 Executive assessment 02
02 Scope and test results 03
03 Cross-account rejection 04
04 Token lifecycle 05
05 Password verification 06
06 Validation and conclusion 07

Executive assessment

01 ACCOUNT MANAGEMENT / CONTROL VALIDATION

Result: No CSRF vulnerability identified in the tested scenarios.

The email-change workflow rejected missing, invalid and cross-account tokens. The current token was accepted repeatedly before a page reload; the old token was rejected afterward and the replacement was accepted. Password-change captures show a success response with the correct current password and rejection with an incorrect current password.

Assessment detail	Recorded scope
Assessor	Haroon Shahid
Target	Anonymized third-party web application
Features	Email change and password change through a shared account-update endpoint
Approach	Manual authenticated request manipulation using Burp Suite Repeater and browser inspection
Test time	7 November 2026 UTC, as shown in captured HTTP response timestamps
Finding classification	No confirmed vulnerability; CVSS not applicable

What the assessment establishes

The tested email-change requests demonstrate token enforcement and rejection of a valid token from another account context. Token reuse alone is not a security finding. The observations are consistent with token rotation on page reload; they do not reveal how tokens are generated or stored.

Evidence and reporting basis

Prepared from the supplied chat transcript, 18 screenshots and the assessor's clarification that both cross-account tokens were valid during the substitution test. This is a retrospective evidence review; no live retesting was performed for this report.

The target, endpoint, accounts and all credentials are omitted from this edition. Included figures contain only original response excerpts. Readable request summaries are separately labeled transcriptions, with symbolic token aliases.

Scope and test results

02 COVERAGE / EVIDENCE STRENGTH

A working request was captured before browser submission, sent to Repeater, and the intercepted browser request was dropped. Tests varied the relevant input while retaining the authenticated context. Page reloads were controlled when examining token reuse and replacement.

Test	Observed result	Source
Email: valid token	Success response	E04
Email: token omitted	Erreur Security 1	E05
Email: empty token	Same security error	Transcript
Email: invalid123	Error CSRF Token	E06
Email: one character modified	Error CSRF Token	Assessor account
Email: Account A + Token B	Error CSRF Token; both tokens valid	E09 + clarification
Email: same token, no reload	Two success responses	E10, E11
Email: old token after reload	Error CSRF Token	E12
Email: replacement after reload	Success response	E14
Password: correct / incorrect current password	Success / current-password error	E16, E17
Email: POST changed to GET	Erreur Security 1	E18
Password: reuse and reload behavior	Reported same behavior as email	Assessor account

Limits of the conclusion

Most outcomes are established by response bodies, not independent post-test login or persisted-state checks. A later browser capture supports the email update. No cross-site browser exploit was demonstrated. Origin/Referer enforcement, SameSite behavior, token entropy, same-user cross-session binding and the full password-CSRF bypass matrix were not established. Findings from email tests are not automatically attributed to password changes.

The supplied program material is a failed page load and a search-result snippet, not a verified scope/disclosure policy. This report makes no claim that program authorization or publication terms were independently verified.

Cross-account rejection

03 WORKING BASELINE / VALID TOKEN SUBSTITUTION

Token aliases: T1 is Account A's captured email token; TB is Account B's different token. Session A denotes Account A's retained authentication context. The aliases replace secrets; they are not token values.

E04 **Baseline accepted**

```
REQUEST SUMMARY | POST [ACCOUNT_UPDATE_ENDPOINT]
Session A; change=email; csrf-token=T1
```

```
HTTP/2 200 OK
Server: nginx
Date: Sat, 07 Nov 2026 06:44:17 GMT
```

```
["success","Your email address has been updated.","success"]
```

Original capture reports that the email address was updated. This establishes a working comparison request.

E09 **Account B token rejected**

```
REQUEST SUMMARY | POST [ACCOUNT_UPDATE_ENDPOINT]
Session A; change=email; csrf-token=TB
```

```
HTTP/2 200 OK
Server: nginx
Date: Sat, 07 Nov 2026 06:49:18 GMT
```

```
["error","Error CSRF Token","error"]
```

The request uses the other account's token and returns Error CSRF Token.

Validity of both tokens

Screenshots 7 and 8 show the separate account forms and token values. The assessor explicitly confirms that both tokens were valid during substitution and neither page was reloaded. Both tokens are recorded as valid at the time of the test. The screenshots do not independently show a same-time successful control request for Account B.

Interpretation

A valid token from the other account context did not authorize Account A's request. This supports binding to the originating account/session context. Distinguishing account binding from session binding would require two sessions of the same account; that property was not tested here.

Token lifecycle

04 REUSABLE BEFORE RELOAD / REPLACEMENT AFTERWARD

T1 denotes the same email token across E10-E12. T2 is the replacement shown after reload. The assessor reports that Session A was retained throughout. Each figure preserves the captured response time and result.

E10 First request accepted

REQUEST SUMMARY | Session A + T1; no page reload

```
HTTP/2 200 OK
Server: nginx
Date: Sat, 07 Nov 2026 06:52:26 GMT
```

```
["success","Your email address has been updated.","success"]
```

E11 Repeated request accepted

REQUEST SUMMARY | Session A + T1; still no page reload

```
HTTP/2 200 OK
Server: nginx
Date: Sat, 07 Nov 2026 06:52:54 GMT
```

```
["success","Your email address has been updated.","success"]
```

E12 Old token rejected after reload

REQUEST SUMMARY | Session A + T1; page reloaded

```
HTTP/2 200 OK
Server: nginx
Date: Sat, 07 Nov 2026 06:56:37 GMT
```

```
["error","Error CSRF Token","error"]
```

E14 Replacement token accepted

REQUEST SUMMARY | Session A + T2; obtained after reload

```
HTTP/2 200 OK
Server: nginx
Date: Sat, 07 Nov 2026 06:58:28 GMT
```

```
["success","Your email address has been updated.","success"]
```

These captures supersede the initial single-use-token hypothesis. At least two requests were accepted with the same token before reload. The observed sequence supports replacement on reload; it does not establish every event that can rotate or expire a token. Reusability alone does not demonstrate CSRF [1].

Password verification

05 CORRECT-PASSWORD BASELINE / INCORRECT-PASSWORD REJECTION

The password form in screenshot 15 contains Current Password, New Password and Confirm New Password. Both password captures use the same shared account-update endpoint with change=pwd. All credential values are excluded from this edition.

E16 Correct current password accepted

```
REQUEST SUMMARY | change=pwd; csrf-token=[REDACTED];
mem_old_pwd=[CORRECT]; new values=[REDACTED]
```

```
HTTP/2 200 OK
Server: nginx
Date: Sat, 07 Nov 2026 07:03:30 GMT
```

```
["success","Your password has been changed.","success"]
```

The application returns Your password has been changed. The capture records a success response; a subsequent login verification is not supplied.

E17 Incorrect current password rejected

```
REQUEST SUMMARY | change=pwd; csrf-token=[REDACTED];
mem_old_pwd=[INCORRECT]; new values=[REDACTED]
```

```
HTTP/2 200 OK
Server: nginx
Date: Sat, 07 Nov 2026 07:04:12 GMT
```

```
["error","The Current Password is not correct. <br>
 Please, indicate the password you have used to login to your current
session in the field 'Current Password\u2019.","error"]
```

The application reports that the current password is not correct. The tab title says Correct Current Password, but the request and response support the incorrect-password test described by the assessor.

Interpretation and boundary

The observed rejection supports enforcement of the current-password check. This adds a barrier to a conventional forged password-change request. It does not establish the order of password and CSRF validation, or prove token validation on the password path.

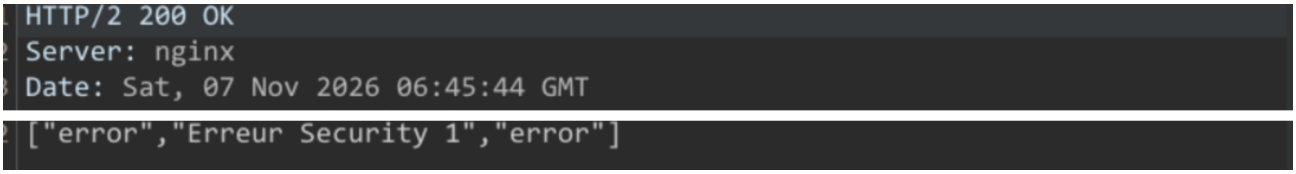
Password token reuse and rejection after reload were reported by the assessor. Dedicated password captures for missing, empty, invalid and cross-account CSRF tokens were not supplied. Empty or omitted current-password tests and session invalidation after password change are also outside the documented coverage.

Validation and conclusion

06 ADDITIONAL REJECTION EVIDENCE / ASSESSMENT CLOSURE

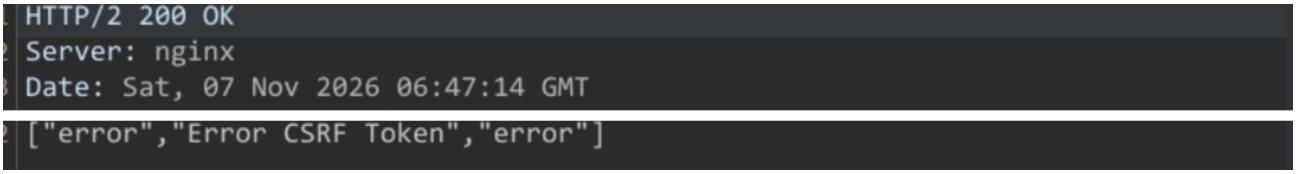
E05 Missing token rejected

REQUEST SUMMARY | change=email; csrf-token omitted



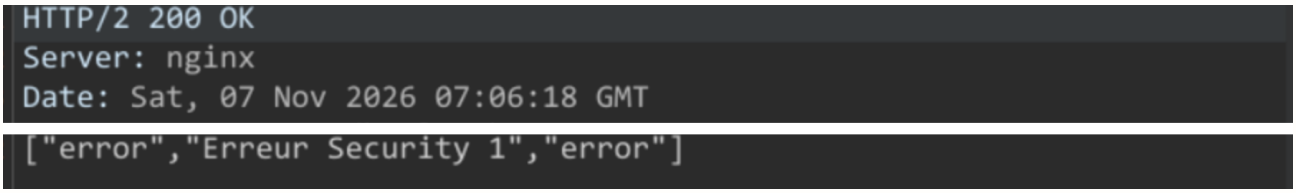
E06 Arbitrary token rejected

REQUEST SUMMARY | change=email; csrf-token=invalid123



E18 Tested GET request rejected

REQUEST SUMMARY | GET [ACCOUNT_UPDATE_ENDPOINT]?change=email&...



The capture moves email parameters into the query string. The assessor identifies the token as valid. The security error proves rejection of this request, not a universal method restriction or a browser-level CSRF result.

Final assessment

No exploitable CSRF or unauthorized state change was demonstrated. No vulnerability-specific remediation or severity score is justified by this evidence. The defensible outcome is a documented validation of the tested controls, with the coverage limits on page 3.

Evidence handling and reference

E-numbers match the original screenshot numbers. Figures retain exact response pixels, enlarged for readability; request panes and unrelated headers are excluded. They are excerpts, not complete HTTP captures. No generated text or synthetic detail is added to the screenshots. The companion manifest records original-file SHA-256 hashes.

[1] OWASP, Cross-Site Request Forgery Prevention Cheat Sheet. Recognizes per-session and per-request token designs. Accessed 7 November 2026.

OWASP CSRF Prevention Cheat Sheet